

Phalanx

Secure Local LLM Serving

Technical Documentation

Ritvik Indupuri

1 Project overview

Phalanx combines a local language-model server with a prompt-screening gateway, explicit token-block memory management and Kubernetes security configuration. The serving application loads Qwen/Qwen2.5-0.5B-Instruct through Hugging Face Transformers and executes real PyTorch forward passes on the CPU.

The gateway checks chat text for specified injection patterns and masks recognized sensitive values before forwarding approved requests. The inference engine tokenizes the prompt, schedules sequences, samples generated tokens and returns an OpenAI-style chat response. Separate modules provide model-file inspection and speculative-token verification.

Subsystem	Implemented responsibility
Gateway	Regex prompt checks, secret redaction, JSON reverse proxy and security counters.
Inference engine	Model loading, prefill and decode execution, temperature and top-p sampling.
Memory and scheduling	Fixed-size block allocation, prefix lookup, waiting/running/preempted queues.
Developer utilities	Model-file scanner, demonstrations, unit checks and benchmark scripts.
Kubernetes	Two-container Deployment, NetworkPolicy, HPA and Kyverno ClusterPolicy.

Model and service boundaries

Qwen is the model-backed component. There is no autonomous planner, tool-calling agent or multi-agent orchestration loop. The agent icon therefore marks the local LLM only. The gateway, scheduler and scanner are deterministic Python components.

The package name `vllm_core` refers to this repository's implementation. It does not import the vLLM serving library. The engine maintains a paged tensor pool, while its current decode path recomputes the full context for each sequence. These are distinct behaviors and are documented separately.

Reading guide

Sections	Coverage
2-4	System architecture, request lifecycle and HTTP contracts.
5-7	Generation, block management, scheduling and security utilities.
8-10	Kubernetes architecture, controls and local setup.
11-12	Telemetry, benchmark interpretation, code references and next steps.

2 System architecture

The Kubernetes manifests place the security gateway and inference engine in the same pod. Deployment controls surround the request path; the inference container owns the local model, scheduler and memory state.

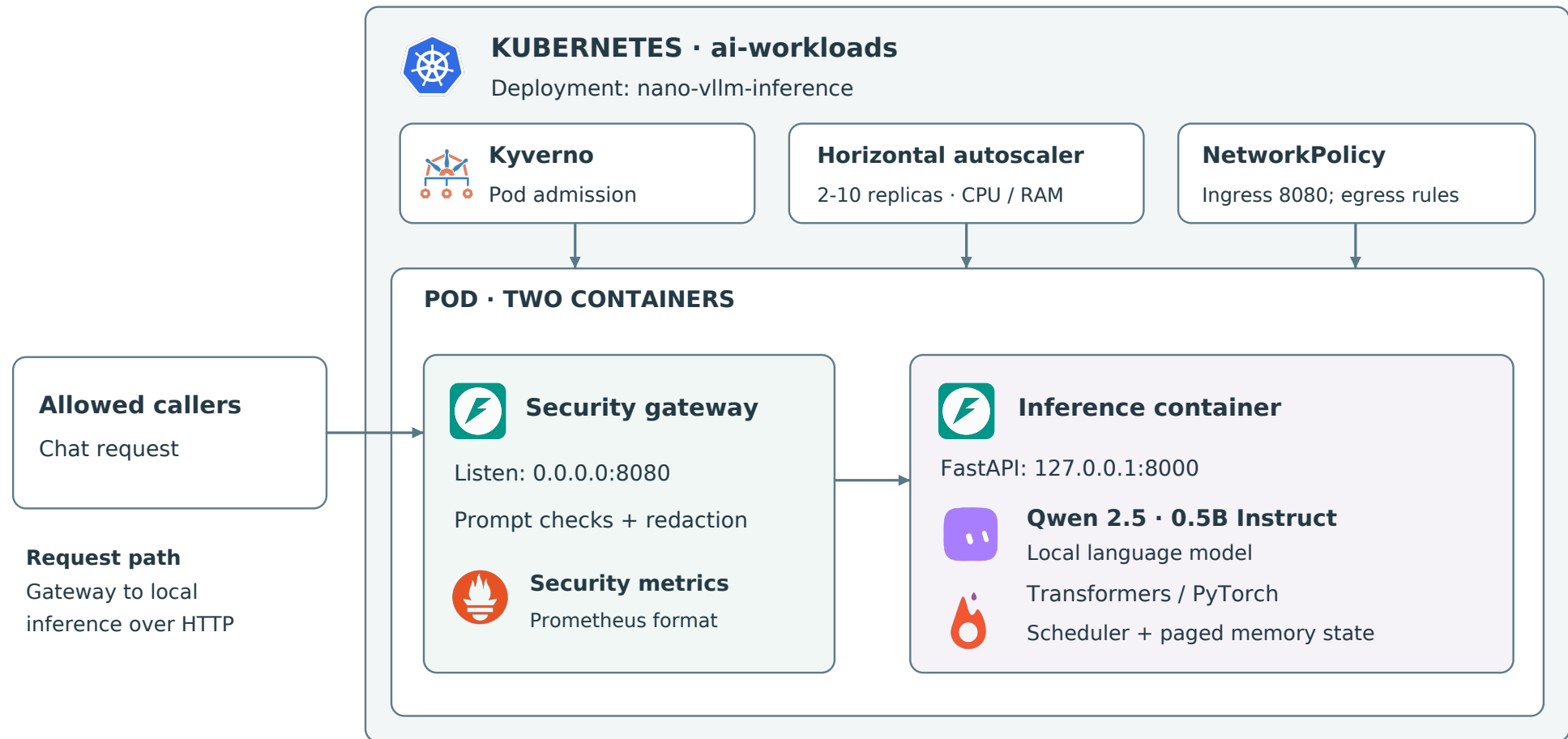


Figure 1: System architecture within the Kubernetes deployment and pod boundaries.

The gateway forwards approved JSON to localhost:8000 through the shared pod network. Kyverno governs pod admission, the HPA scales the Deployment, and NetworkPolicy selects its pods. The default is two replicas. Prometheus denotes the gateway metrics format. Section 8 details storage and network controls; the manifests require working images and deployment prerequisites.

3 LLM request lifecycle

A request passes through deterministic screening before model execution. Injection matches terminate at HTTP 403. Approved messages are sanitized and forwarded to the inference API.

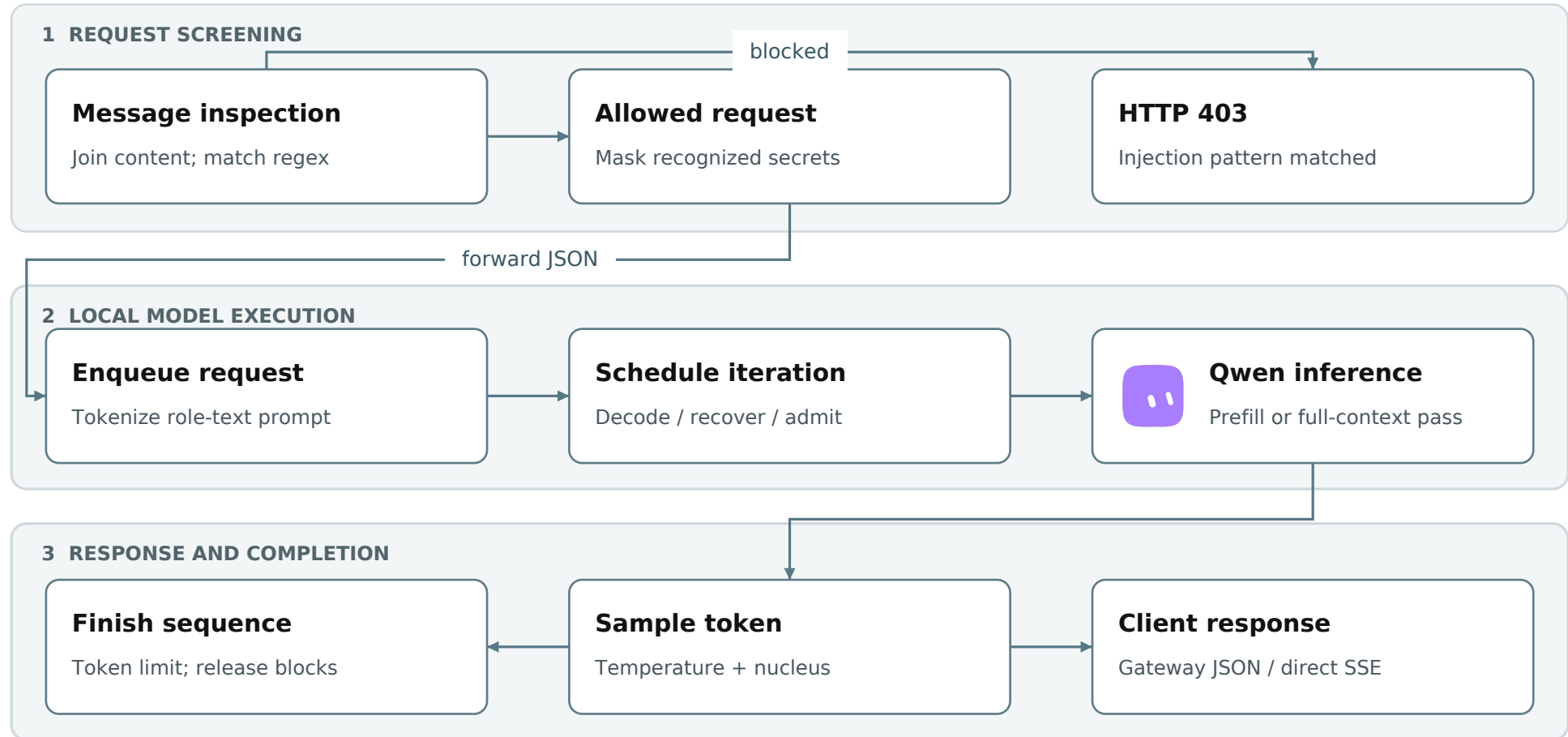


Figure 2: Screening, scheduled generation and response delivery.

Scheduling and sampling repeat until the requested token budget is reached. Completion releases the sequence's blocks. The gateway returns a complete JSON response; SSE is implemented by the direct inference endpoint. Model icons identify LLM execution, not the surrounding control logic.

4 HTTP interfaces and request handling

Service and route	Contract
Gateway POST /v1/chat/completions	Chat messages, max_tokens 1-512 (default 32), temperature 0.7, top_p 0.9 and stream=false by default.
Gateway GET /healthz	Returns healthy status and the ai-security-gateway service name.
Gateway GET /metrics	Prometheus text exposition for four process-local counters.
Engine POST /v1/chat/completions	max_tokens 1-1024 (default 64); temperature 0-2; top_p 0-1; JSON or direct SSE.
Engine GET /v1/models	Reports the loaded model name in an OpenAI-style list.
Engine GET /metrics	JSON memory, prefix-cache and scheduler statistics; 503 while uninitialized.

Prompt construction and model selection

Both services accept messages containing role and content strings. The gateway checks a space-joined view of all content. The engine constructs newline-separated role: content entries and appends assistant:. It does not apply the tokenizer's model-specific chat template.

Startup explicitly loads Qwen/Qwen2.5-0.5B-Instruct. The request model field is echoed in completion responses but does not choose or load a different model. The gateway's default model string is gpt2, so clients should send the Qwen identifier explicitly for consistent response metadata.

Responses and forwarding

Non-streaming responses contain an ID, creation timestamp, assistant message, usage counts and system_telemetry. finish_reason is always stop. A matching injection returns 403 with security_policy_violation and PROMPT_INJECTION_DETECTED. An upstream connection failure becomes 503. Forwarding uses HTTPX with a 60-second timeout.

The direct engine stream emits data: JSON chunks followed by data: [DONE]. Its loop calls engine.step(), decodes the latest output token and yields it. The gateway always parses the upstream body as JSON; use stream=false through that path. Direct SSE is a separate interface, not end-to-end streaming through the proxy. It reads the latest token without a per-request emission cursor, so this interface should not be described as exactly-once streaming under concurrent handler interleaving.

Execution ownership

Each HTTP handler drives the shared engine until its request finishes, yielding briefly between iterations. Model calls are synchronous inside those iterations. This differs from a dedicated background inference worker or a fused tensor batch. Engine readiness checks return 503 when the global instance is absent.

5 Model execution and sampling

Loading and initialization

NanoLLMEngine loads AutoTokenizer and AutoModelForCausalLM from the selected model path. It assigns EOS as the padding token when needed, loads float32 weights with `low_cpu_mem_usage` enabled, moves the model to the selected device and enters evaluation mode. HTTP startup fixes the device to cpu, 256 blocks, 16 tokens per block and a maximum running batch of 16.

Architecture dimensions come from model configuration: layer count, key/value-head count and head dimension, with fallback names for GPT-style configurations. Separate K and V tensors have shape [layers, blocks, KV heads, block size, head dimension]. For float32, total pool bytes equal $2 \times \text{layers} \times \text{blocks} \times \text{KV heads} \times \text{block size} \times \text{head dimension} \times 4$.

Prefill and decode

New requests are encoded and placed in the scheduler's waiting queue. Prefill sends the complete prompt to the model with `use_cache=True`, samples from the final-position logits and writes returned K/V values into the sequence's assigned physical blocks. Each token offset maps to block index offset `// block_size` and slot offset `% block_size`.

Decode reconstructs `prompt_tokens + output_tokens` and calls the model with `use_cache=False` for each scheduled sequence. It samples the final-position logits and appends one output token. The allocated paged K/V pool is not read by an attention kernel in this decode path. Requests are iterated individually rather than combined into one model-forward tensor batch.

Sampling and termination

Temperatures at or below 0.0001 select argmax. Otherwise logits are divided by temperature and converted with softmax. Probabilities are sorted, cumulative mass is compared with `top_p`, the threshold mask is shifted to retain the crossing token, and the retained distribution is renormalized before multinomial sampling.

A sequence finishes when generated-token count reaches `max_new_tokens` or its status is `FINISHED`. There is no EOS-based early-stop branch. Completion frees its block table and records a timestamp. Generated tokens and request metadata remain in `requests_map`.

Timing semantics

`first_token_time` is set immediately before the first prefill forward pass. The reported `ttft_ms` therefore measures admission-to-prefill timing rather than the complete time until the first sampled token is available. Step latency covers scheduling and all forward passes performed in that iteration; it is not a per-request inter-token latency measurement.

6 Memory management and scheduling

Physical pool and sequence tables

MemoryPool maintains free IDs, allocated IDs and PhysicalBlock records. Each block tracks allocation state, reference count and an optional prefix hash. A BlockTable maps a request's logical block sequence to physical IDs and records its token count. New allocations need $\text{ceil}(\text{prompt_length} / \text{block_size})$ blocks, less any supplied shared capacity.

Appending a slot reuses remaining capacity in the final block or allocates another block. Shared IDs increment reference counts, and freeing a sequence decrements each referenced block. This implements shared-block accounting; no write-triggered copy operation is present.

Prefix lookup

PrefixCache is a nested dictionary trie keyed by complete block-sized token tuples. Lookup follows the longest matching prefix and returns physical IDs plus the matched-token count. Insertion stores full chunks only. Nodes track access time and access count; statistics expose cached blocks, hits, misses and hit rate.

The scheduler uses the match to select shared block IDs and reduce admission-budget accounting. The model still performs full-prompt prefill. Prefix nodes are not invalidated when sequence blocks are freed, and the cache has no independent ownership retention or eviction routine. Treat its hit rate as lookup telemetry rather than proof of avoided forward-pass work.

Iteration ordering

Phase	Scheduling behavior
Running decode	Finish completed entries; reserve one slot for others; preempt a request when no slot is available.
Preempted recovery	Try FIFO recovery before new work, while capacity exists and remaining budget exceeds 32. Allocate total context length and resume decode.
New admission	Inspect the waiting head, check prefix match, step budget and free blocks; allocate the prompt and append it to the running queue.

Default scheduling budget is 2,048 tokens per step. Each decode uses one budget unit; a new prompt is charged its unmatched prefix length. A head request that cannot fit stops admission for that iteration. `chunked_prefill_size` is stored with a default of 512, but admission does not split a prompt into chunks.

Preemption preserves prompt and output tokens in the request object while releasing its blocks. Recovery can recompute the full context through the existing decode path. Counters record completion and preemption events. Memory statistics describe allocation bookkeeping, not independently measured accelerator memory or serving capacity.

7 Prompt controls and model-file inspection

Gateway screening

`evaluate_prompt_safety` applies case-insensitive regexes for instruction overrides, disregarded constraints, selected persona hijacks, system-prompt extraction, base64 decode-and-execute wording and an HTML script opener. All matching injection labels are returned. An injection match blocks immediately, before secret masking.

Recognized sensitive value	Replacement label
AWS access-key ID: AKIA + 16 uppercase letters/digits	[REDACTED_AWS_ACCESS_KEY_ID]
GitHub token: ghp_ + 36 alphanumeric characters	[REDACTED_GITHUB_PERSONAL_ACCESS_TOKEN]
Email-address pattern	[REDACTED_EMAIL_ADDRESS]
US SSN: three-two-four digits	[REDACTED_US_SOCIAL_SECURITY_NUMBER]

Sensitive matches alone are allowed. With masking enabled, matching values are replaced and a finding records each category and its match count. The gateway then re-evaluates each message separately and forwards the sanitized content. This is pattern-based screening; it does not use a classifier or inspect generated responses.

Standalone model scanner

`scan_model_file` returns filepath, format, size in MiB, `is_safe`, `risk_level` and findings. For a .safetensors filename, an eight-byte prefix is interpreted as an unsigned little-endian header length and produces a LOW result. The scanner does not parse the header JSON or validate tensor offsets and file consistency.

For .bin, .pt, .pth and .pkl, the initial risk is MEDIUM. SafeModelUnpickler attempts deserialization and intercepts selected dangerous globals from os, subprocess, sys, builtins, socket, posix and nt. A recognized callable makes the result unsafe and CRITICAL. Other deserialization exceptions are ignored.

This utility invokes an unpickler and allows other globals through `find_class`; it is not a non-executing sandbox or a complete model-safety certificate. Unknown extensions retain the initial LOW classification. Use it only with controlled demonstration files. It is called by security demonstrations and tests, not by the serving startup path.

Security separation

Prompt checks protect the gateway route, while model inspection is a separate developer operation. Kubernetes admission and network configuration add deployment controls. None of these components provides application identity authentication, output moderation or an automatic scanner gate before model loading.

8 Kubernetes deployment architecture

The manifests describe a Deployment in ai-workloads with a gateway sidecar and local inference container. Official Kubernetes and Kyverno icons identify orchestration and admission control.

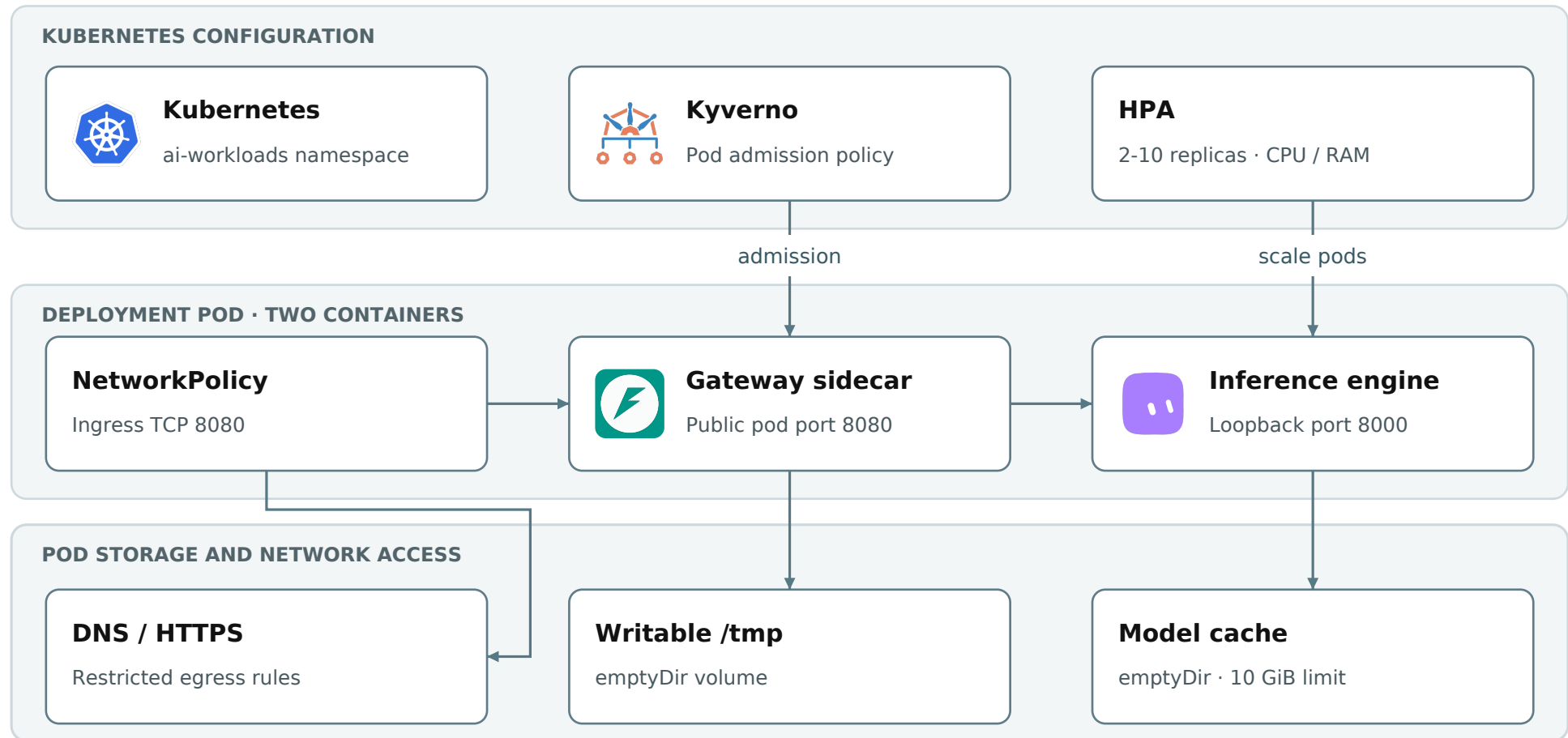


Figure 3: Pod boundaries, deployment controls and writable storage.

The two containers share the pod network, allowing the gateway to reach localhost:8000. The inference container binds loopback. Kubernetes and HPA boxes describe resources and control relationships, not additional request-processing services. The image names are deployment placeholders to replace with built images.

9 Deployment controls and resources

Resource	Declared configuration
Deployment	nano-vllm-inference; two replicas; app label nano-vllm-inference.
Gateway container	CPU request/limit 500m/1000m; memory 512Mi/1Gi; port 8080.
Inference container	CPU request/limit 2000m/4000m; memory 4Gi/8Gi; loopback port 8000.
Pod identity	UID/GID/fsGroup 10001; runAsNonRoot; RuntimeDefault seccomp.
Container restrictions	Read-only root filesystem, no privilege escalation and drop ALL capabilities.
Storage	emptyDir for /tmp; separate /tmp/huggingface cache with 10Gi size limit.

Traffic policy

NetworkPolicy selects the inference pods and governs ingress and egress. It permits TCP 8080 from the ingress-nginx namespace OR pods labeled as api-gateway in the workload namespace. These are alternative peer entries, not a requirement that both selectors match.

DNS is allowed on UDP/TCP 53 to kube-dns pods in kube-system. HTTPS egress is allowed to IPv4 destinations except 169.254.169.254 and the 10/8, 172.16/12 and 192.168/16 ranges. These are CIDR controls, not a hostname allowlist for model registries. Enforcement depends on the cluster network implementation.

Admission and autoscaling

The Kyverno ClusterPolicy runs in Enforce mode with background checks. It matches pods in ai-workloads and validates non-root execution, container read-only roots, no privilege escalation, dropped capabilities and absence of hostPath volumes. It does not verify image signatures or inspect model files.

The HPA targets the Deployment, with 2-10 replicas, 75% CPU utilization and 80% memory utilization targets. Scale-up has a zero-second stabilization window and allows 100% growth per 15 seconds. Scale-down uses a 300-second window and 20% reduction per 60 seconds. These resource metrics do not consume the gateway's security counters.

Deployment preparation

Provide the namespace, working container images, Kyverno, resource metrics and a policy-capable network plugin. The repository has no Dockerfile, Service or Ingress manifest. Gateway probes check /healthz; the engine has no readiness probe. MODEL_NAME, NUM_KV_BLOCKS and BLOCK_SIZE are declared in YAML, but server startup uses hardcoded values. HF_HOME supplies a writable model-cache location.

10 Local installation and use

Run from the repository root in an isolated Python environment. The requirements specify minimum versions rather than a locked dependency set. HTTPX is imported by the gateway and requests by network clients; install them explicitly alongside the supplied requirements.

```
git clone https://github.com/ritvikindupuri/Phalanx.git
cd Phalanx
python -m venv .venv
# Activate .venv for your operating system.
python -m pip install -r requirements.txt
python -m pip install httpx requests
```

Start both applications

Start the engine first and allow model loading to finish. Use a separate terminal for the gateway. These commands invoke the actual ASGI applications; neither module implements a custom command-line server launcher.

```
python -m uvicorn server.app:app \
--host 127.0.0.1 --port 8000

python -m uvicorn gateway.proxy:app \
--host 127.0.0.1 --port 8080
```

The gateway's `UPSTREAM_ENGINE_URL` is fixed to `http://localhost:8000`. Initial model loading requires access to the model repository or a populated local cache. The default application uses CPU inference and float32 weights.

Submit a chat request

```
curl http://127.0.0.1:8080/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{"model": "Qwen/Qwen2.5-0.5B-Instruct",
"messages": [{"role": "user",
"content": "Explain virtual memory."}],
"max_tokens": 32, "stream": false}'
```

Direct engine demonstration

```
python demo.py --model Qwen/Qwen2.5-0.5B-Instruct \
--blocks 256 --block-size 16
```

`demo.py` instantiates the engine directly and prints scheduled outputs and memory statistics. It does not pass requests through the HTTP gateway. `live_network_client.py` and `live_qwen_test.py` instead call running services. `test_qwen_infra.py` combines local file inspection, direct guardrail calls and direct Qwen execution; those are distinct demonstration paths.

11 Telemetry and evaluation utilities

Gateway counters

Prometheus name	Increment semantics
ai_gateway_requests_total	One per validated chat request handled.
ai_gateway_injections_blocked_total	One per blocked request.
ai_gateway_pii_sanitized_total	Number of sensitive categories detected in the joined prompt, not number of individual values.
ai_gateway_forwarded_to_inference_total	One before an upstream attempt; includes attempts that fail.

Counters live in a process dictionary and reset on restart. `/healthz` and `/metrics` calls do not increment the chat request counter. The Deployment's scrape annotations point to gateway port 8080. The engine endpoint returns JSON instead of Prometheus exposition.

Standalone speculative verifier

SpeculativeVerifier accepts caller-supplied draft tokens and draft/target probability arrays. Each proposed token is accepted with $\min(1, \text{target_probability} / \max(1e-9, \text{draft_probability}))$. At the first rejection, it samples from normalized $\max(0, \text{target} - \text{draft})$, falling back to target argmax if residual mass is negligible. If every draft is accepted, it can sample one bonus target token.

The utility counts inspected draft tokens, accepted drafts and verifier calls. Its `speedup_ratio` is accepted drafts per call plus one. It is not wall-clock speedup. `gamma` and `temperature` do not drive a model call in `verify_tokens`. No draft model or speculative verifier is connected to `NanoLLMEngine.step()`.

Benchmark interpretation

Script	What the code measures
benchmark_memory.py	Seeded random sequence lengths; block capacity versus fixed maximum reservation. Reported concurrency is a calculated estimate.
benchmark_speculative.py	Synthetic NumPy distributions with <code>gamma 2-6</code> ; acceptance and tokens-per-verifier-call statistics.
benchmark_serving.py	Real CPU model execution (default <code>gpt2</code>), aggregate token throughput, step timing and request timing percentiles.

The test files cover block allocation/refcounts, prefix matching, scheduler lifecycle, prompt checks, redaction and a constructed malicious-pickle case. `run_tests.py` invokes seven test functions and reports failures. `security_demo.py` also prints a static Kubernetes control summary; it does not apply or validate cluster resources. These are supplied evaluation tools, not benchmark results established by this documentation review.

12 Code references and next steps

The implementation descriptions above were checked against the source and manifests at the following revision. The review concerns code behavior and structure; it does not assert measured performance or successful runtime deployment.

Repository: [ritvikindupuri/Phalanx](https://github.com/ritvikindupuri/Phalanx)

Revision: 3ba860957d798a5bdaae5eaed9be13672e6a0094

Area	Source files
HTTP and prompt security	gateway/proxy.py; gateway/guardrails.py; server/app.py
Model execution	vllm_core/engine.py
State and scheduling	vllm_core/block_manager.py; prefix_cache.py; scheduler.py
Speculative verification	vllm_core/speculative.py
Model inspection	security/model_scanner.py
Cluster configuration	k8s/deployment.yaml; network-policy.yaml; hpa.yaml; kyverno-policy.yaml
Demonstrations	demo.py; security_demo.py; test_qwen_infra.py; live_network_client.py; live_qwen_test.py
Evaluation	benchmarks/benchmark_memory.py; benchmark_speculative.py; benchmark_serving.py; tests/; run_tests.py
Dependencies and context	requirements.txt; README.md; TECHNICAL_DOCUMENTATION.md

Next steps

A next serving iteration can connect decode to reusable K/V state, establish cache ownership and eviction, implement real chunked prefill and move scheduling into a single background worker. End-to-end streaming should track per-request token cursors and relay SSE through the gateway. Configuration loading, complete deployment packaging and engine readiness would make the local and cluster paths easier to operate consistently.

Model inspection can move to non-executing format validation and explicit artifact provenance checks. Performance evaluation can separate queue delay, first-token completion, per-token latency and allocation efficiency, with results tied to a fixed model, hardware configuration and workload.

Diagram symbols

Official FastAPI and PyTorch artwork identifies the application framework and tensor runtime. Official CNCF artwork identifies Kubernetes, Kyverno and Prometheus. The purple icon from the supplied agent pack marks local LLM execution. The Prometheus icon denotes the metrics format and external scraping integration; no Prometheus server manifest is included.

Source links and icon origins are also included with the editable documentation source. Diagrams use vector text and connectors, with short labels placed on opaque backgrounds. No documentation date is printed.